# Kovex: Graph-Aware, Per-Node Coordination for Multi-Agent LLM Coding Systems

Ibrahem Alkurdi

ibrahem.alkurdi@gmail.com

May 10, 2026

**Abstract**

Multi-agent LLM coding systems operating concurrently on shared codebases face a systematic failure mode that neither message-passing coordination nor text-level concurrency control can address: *semantic conflicts*, arising when one agent's write to a function signature or interface boundary invalidates assumptions held by a dependent agent. We present **Kovex**, a coordination substrate that indexes a live codebase into a Neo4j property graph, exposes it to agents via a Model Context Protocol (MCP) server, and notifies registered dependents via bounded breadth-first search over the call graph when a write is committed. We evaluate Kovex in two stages. A systems correctness evaluation on 14 tasks confirms notifications fire at the correct call-graph distances ($d \in \{0, 1, 2\}$) with 100% accuracy and mean latency 45.6 ms, while correctly suppressing notifications at $d = 3$ ($k = 2$ false-negative rate 14.3%). A real agent effectiveness experiment using Claude Sonnet 4.6 as both agents (50 trials, $n = 5$ per cell) shows that Kovex-coordinated agents achieve an 80% final-code pass rate on tasks within the notification radius versus 0% for uncoordinated baseline agents, with two tasks reaching 100% vs. 0%. The T13 false-negative control correctly collapses to near-baseline (20% vs. 0%, unattributable to Kovex), validating the CALLS-only BFS design.

## 1 Introduction

The deployment of large language models as collaborative coding agents has advanced rapidly, yet a fundamental coordination problem has emerged: when two agents operate concurrently on a shared codebase, one agent's modification to a function signature or type definition can silently invalidate the code being generated by the

other. CooperBench [11] quantifies this as the *curse of coordination*, reporting a 30% lower success rate for collaborative agent pairs compared to single-agent execution. CodeCRDT [17] demonstrates that text-level concurrency control, while guaranteeing syntactic convergence, still produces a 5–10% semantic conflict rate. Existing frameworks resolve concurrency by eliminating it (MetaGPT [9], ChatDev [18]) or by deferring to global test re-execution (AgentForge [13]).

We identify the root cause as a missing representation layer. No system maintains a persistent, queryable model of the codebase's dependency structure through which agents can register read dependencies and receive targeted notifications when those dependencies change. Without this layer, verification is either absent or exhaustive.

We contribute **Kovex**, comprising: (1) a Neo4j property graph indexed from live source code; (2) an MCP server through which agents register read dependencies on named graph nodes; and (3) a write-triggered bounded BFS notification algorithm that delivers targeted re-verification directives to registered dependents within a configurable $k$-hop call-graph radius. Our central research question is:

> *Does graph-aware, per-node read/write coordination with write-triggered targeted verification reduce semantic conflict rates in multi-agent coding systems, relative to uncoordinated baselines?*

We answer this with two complementary evaluations: a systems correctness study confirming the coordination mechanism operates as specified, and a real agent effectiveness experiment with LLM agents demonstrating a measurable downstream reduction in semantic conflict rates.

# 2 Related Work

## 2.1 Multi-Agent LLM Systems for Software Engineering

MetaGPT [9] encodes Standardized Operating Procedures into prompt sequences, assigning specialized roles that hand off work through structured artifacts. ChatDev [18] implements a similar role hierarchy through natural-language dialogue. AutoGen [23] generalizes this into a flexible multi-agent conversation framework. All three enforce coordination sequentially; none maintains a runtime dependency model of the codebase. SWE-agent [25] focuses on single-agent performance on SWE-bench [10];

SWE-EVO [21] extends evaluation to long-horizon tasks requiring coordinated modifications across an average of 21 files, where the strongest agents achieve only 21% resolution.

CooperBench [11] provides the most direct empirical characterization of the coordination problem. Across 652 collaborative coding tasks, agent pairs achieve 30% lower success rates than single agents, with communication reducing syntactic merge conflicts but failing to prevent semantic conflicts arising from interface incompatibilities. AgentForge [13] advances the state of the art by binding verification to each state transition via sandboxed execution, achieving 40.0% on SWE-bench Lite. However, its verification is global and cannot determine from the codebase's dependency structure which agents need to re-verify.

## 2.2 Graph-Based Code Intelligence

Ferrante et al. [6] introduced the Program Dependence Graph, unifying control and data dependencies for each program operation. Yamaguchi et al. [24] unified the AST, CFG, and PDG into the Code Property Graph stored in a graph database, demonstrating that code structure can serve as a first-class database artifact rather than a transient in-memory representation. ACER [3] provides a language-agnostic pipeline using Tree-sitter [22] to extract call graphs at scale. GraphRAG [4] and its multi-agent extension [8] show that LLMs can reason effectively over graph-structured knowledge. None of these uses a code graph as a live write-aware coordination layer.

## 2.3 Concurrency Control and Collaborative Editing

Ellis and Gibbs [5] introduced Operational Transformation for convergent collaborative text editing. Shapiro et al. [20] formalized CRDTs, guaranteeing strong eventual consistency without locking. Pugachev [17] applies CRDTs to multi-agent LLM code generation, achieving 100% syntactic convergence across 600 trials while reporting a residual 5–10% semantic conflict rate. This establishes that text-level consistency is strictly weaker than the coordination guarantee Kovex targets. Gray et al. [7] analyze lock granularity trade-offs; Kovex adopts an analogous design at semantic node granularity.

### 2.4 Verification, Semantic Conflicts, and Interface Contracts

Meyer's Design by Contract [15] established function signatures as contractual obligations between implementors and callers. Krieter et al. [12] show that higher-order semantic merge conflicts are common in open-source projects even without concurrent agents. Santos et al. [19] and Barbosa et al. [2] propose post-hoc static analysis for detection. Kovex targets the same failure class but prevents conflicts at coordination time. Consumer-driven contract testing [1, 14] provides a structural analogy: a read-registration is equivalent to a consumer contract, and write-triggered notification is the automated provider responsibility mechanism.

## 3 System Design

### 3.1 Overview

Kovex comprises three components: the *Kovex Graph* (a Neo4j labeled property graph indexed from live source code), the *MCP Server* (a Model Context Protocol server exposing the graph to agents as a tool API and maintaining a read registry), and the *Write Log* (an append-only record of every write event and its notifications).

### 3.2 Graph Schema

The graph models the *semantic contract surface* of a codebase. Four node labels are defined: `Function` (properties: `name`, `file`, `signature`, `return_type`, `body_hash`), `Type`, `Module`, and `Symbol`. Seven edge types encode semantic dependencies: CALLS, USES_TYPE, DEFINES, IMPORTS, INHERITS, IMPLEMENTS, and OVERRIDES. The graph is indexed using Tree-sitter [22] for language-agnostic AST extraction.

### 3.3 Read/Write State Model

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the Kovex graph and $\mathcal{A}$ the set of active agents. Each agent $a$ maintains a read set $\mathrm{RS}(a) \subseteq \mathcal{V}$. The global read registry is $\mathcal{R} = \{(a, v, t_r, h_r) \mid v \in \mathrm{RS}(a)\}$. A write by agent $a$ to node $v$ is *observable* iff $h_{\mathrm{post}}(v) \neq h_{\mathrm{pre}}(v)$; non-observable writes do not trigger notifications.

## 3.4 Write-Triggered Notification Algorithm

**Definition 1: $k$-Hop Notification Radius**

Let $d_{\mathcal{G}}(u, v)$ be the shortest-path distance restricted to CALLS edges. The *$k$-hop dependency neighborhood* of $v$ is $\mathcal{N}_k(v) = \{u \in \mathcal{V} \mid d_{\mathcal{G}}(v, u) \leq k\}$. Agent $a$ is a notification target for a write to $v$ iff $\mathrm{RS}(a) \cap \mathcal{N}_k(v) \neq \emptyset$.

**Edge type selection.** Restricting BFS to CALLS edges ensures notifications fire strictly on caller/callee relationships. An all-edge-type BFS places every co-module function pair at $d \leq 2$ via DEFINES edges, conflating module co-location with semantic dependency. Algorithm 1 gives the formal procedure.

**Algorithm 1** Write-Triggered Notification (NOTIFYDEPENDENTS)

**Require:** Graph $\mathcal{G}$, written node $v$, radius $k$, registry $\mathcal{R}$
**Ensure:** Notification triples $(a, u, d)$

```
frontier ← {v};  visited ← {v};  notifs ← ∅
for d = 1 to k do
    next ← ∅
    for each u ∈ frontier do
        for each w ∈ CallNeighbors(u, G),  w ∉ visited do
            visited += w;  next += w
            for each (a, w, t_r, h_r) ∈ R do
                notifs += (a, w, d)
            end for
        end for
    end for
    frontier ← next
end for
return notifs
```

## 3.5 MCP Server Interface

Eight tools are exposed: `register_read`, `deregister_read`, `list_reads`, `commit_write` (applies patch, runs Algorithm 1, logs event, returns notify set), `get_node` (ephemeral read; no registry side effect), `get_dependents`, `query_graph` (read-only Cypher), and `poll_directives`. The distinction between `get_node` and `register_read` is a deliberate interface contract: ephemeral reads do not enroll the agent in the notification protocol.

### 3.6 Semantic Conflict Definition

**Definition 2: Semantic Conflict**

Let $\text{Valid}(C, \mathcal{G}_t)$ denote that code $C$ type-checks against codebase state $\mathcal{G}_t$. A **semantic conflict** between agents $A$ and $B$ with respect to node $v$ at time $t_w$ holds iff:

(SC1) Agent $A$ commits an observable write to $v$ at $t_w$.

(SC2) $(B, u, t_r, h_r) \in \mathcal{R}$ for some $u \in \mathcal{N}_k(v)$, $t_r < t_w$.

(SC3) Agent $B$ produced code $C_B$ in $[t_r, t_w)$.

(SC4) $\text{Valid}(C_B, \mathcal{G}_{t_w^-}) \wedge \neg\,\text{Valid}(C_B, \mathcal{G}_{t_w^+})$.

## 4 Evaluation

### 4.1 Codebase and Graph Statistics

We index the httpx Python HTTP client library [16]. The resulting Kovex graph contains 550 nodes (358 `Function`, 85 `Type`, 84 `Symbol`, 23 `Module`) and 916 edges (527 Defines, 225 Uses_Type, 99 Calls, 56 Inherits, 9 Imports).

### 4.2 Stage 1: Systems Correctness Evaluation

We design 14 evaluation tasks organized by CALLS-chain distance $d$ between write target and Agent B's registration node, covering three distance groups and a false-negative control group. Each task applies a realistic signature change (parameter removal, renaming, or addition) and verifies Definition 2 condition (SC4) via `mypy`. Table 1 summarizes all tasks.

All 14 tasks pass every applicable check: 12/12 tasks at $d \leq 2$ receive exactly one directive with the correct distance label; T13 and T14 ($d = 3 > k = 2$) receive zero directives. All 14 baseline runs receive zero directives. Table 2 reports notification latency by distance group.

All 12 notifications arrive within 500 ms (maximum 81.8 ms). Mean latency does not increase monotonically with $d$, reflecting that Neo4j query overhead dominates BFS depth at this codebase scale.

Table 1: Evaluation tasks. $d$: CALLS-chain distance. FN: false-negative control.

| Task | Write Target | B Registration | Change |
|---|---|---|---|
| T01 | `AsyncClient.send` | `AsyncClient.send` | param remo |
| T02 | `AsyncClient.request` | `AsyncClient.request` | param rena |
| T03 | `AsyncClient.send` | `AsyncClient.request` | param remo |
| T04 | `AsyncClient.send` | `AsyncClient._send_handling_auth` | param remo |
| T05 | `AsyncClient.request` | `AsyncClient.get` | param rena |
| T06 | `AsyncClient.request` | `AsyncClient.post` | param rena |
| T07 | `AsyncClient._send_handling_redirects` | `AsyncClient._send_handling_auth` | param addi |
| T08 | `AsyncClient._send_single_request` | `AsyncClient._send_handling_redirects` | param addi |
| T09 | `AsyncClient.send` | `AsyncClient.get` | param remo |
| T10 | `AsyncClient.send` | `AsyncClient.post` | param remo |
| T11 | `AsyncClient._send_handling_redirects` | `AsyncClient.send` | param addi |
| T12 | `AsyncClient._send_single_request` | `AsyncClient._send_handling_auth` | param addi |
| T13 | `AsyncClient._send_single_request` | `AsyncClient.send` | param addi |
| T14 | `AsyncClient._transport_for_url` | `AsyncClient._send_handling_auth` | param addi |

## 4.3 Stage 2: Real Agent Effectiveness Experiment

**Setup.** To measure whether Kovex notifications causally improve agent code quality, we run a real two-agent experiment using the Anthropic Claude Sonnet 4.6 model as both Agent A (writer) and Agent B (reader). We select five representative tasks: T01 ($d = 0$), T03 ($d = 1$), T05 ($d = 1$), T09 ($d = 2$), and T13 ($d = 3$, false-negative control). Each task is run $n = 5$ times per condition (50 trials total).

**Protocol.** In the *Kovex condition*, Agent B's Phase 1 session calls `register_read` on its registration node, queries the current signature via `get_node`, and generates initial code. Agent A then calls `commit_write`. Agent B's Phase 2 session—initialized with Phase 1 context—calls `poll_directives` to receive any Kovex notification, calls `get_node` to read the updated signature, and revises its code. In the *baseline condition*, Agent B does not call `register_read` or `poll_directives` and submits its Phase 1 code unchanged after Agent A's write. The metric is `mypy` pass rate on Agent B's final code against the post-write codebase, filtered to errors in Agent B's file only (`agent_code_post_passed`).

Table 2: Notification latency by distance group ($k = 2$, CALLS-only BFS).

| **Group** | $n$ | **Mean (ms)** | **Range (ms)** | **Std (ms)** |
|---|---|---|---|---|
| $d = 0$ | 2 | 39.2 | [34.0, 44.3] | 7.3 |
| $d = 1$ | 6 | 47.3 | [27.9, 81.8] | 21.2 |
| $d = 2$ | 4 | 46.2 | [32.9, 65.5] | 14.7 |
| All | 12 | 45.6 | [27.9, 81.8] | 17.4 |

**Results.** Table 3 reports per-task pass rates across five runs.

Table 3: Real agent effectiveness results ($n = 5$ per cell, Claude Sonnet 4.6). Kovex (notified) vs. Baseline (uncoordinated). T13 is the false-negative control: Kovex correctly issues no notification ($d = 3 > k = 2$).

| **Task** | **Group** | **Kovex Pass** | **Baseline Pass** | **Δ** |
|---|---|---|---|---|
| T01 | $d = 0$ | 4/5 (80%) | 0/5 (0%) | +80pp |
| T03 | $d = 1$ | 5/5 (100%) | 0/5 (0%) | +100pp |
| T05 | $d = 1$ | 4/5 (80%) | 0/5 (0%) | +80pp |
| T09 | $d = 2$ | 3/5 (60%) | 0/5 (0%) | +60pp |
| T01–T09 aggregate | $d \leq 2$ | 16/20 (80%) | 0/20 (0%) | +80pp |
| T13 | $d = 3$ (FN) | 1/5 (20%)* | 0/5 (0%) | ≈0pp |

* All 5 T13 Kovex trials confirmed `notify_set=[]` — the pass is unattributable to Kovex (agent wrote compatible code by chance).

On the 20 trials where Kovex is expected to notify ($d \leq 2$), coordinated agents pass at 80% vs. 0% for uncoordinated baseline agents — a gap of 80 percentage points. T03 and T05 achieve a perfect 5/5 vs. 0/5 split. T09 ($d = 2$) achieves 60% vs. 0%, demonstrating that the two-hop traversal produces actionable directives even at transitive distance.

The T13 false-negative control validates the CALLS-only BFS design: Kovex issues zero notifications across all 5 runs ($d = 3 > k = 2$), and the Kovex pass rate of 20% is within noise of the 0% baseline, confirming that Agent B does not benefit from coordination when Kovex correctly withholds the notification.

**Failure analysis.** The remaining 4/20 Kovex failures are attributable to CLI tool-protocol brittleness: the model occasionally emits a `<tool_call>` block and a `<python_code>` block in the same turn; truncation discards the code block, and the subsequent turn treats the (now absent) code as final. This failure mode is independent of Kovex's notification mechanism and would be eliminated by native SDK tool use, which enforces strict turn structure. The 80% figure is therefore a conservative lower bound on Kovex's effectiveness under a reliable tool-use protocol.

# 5 Discussion

**Edge type selection as a design parameter.** An initial implementation used all edge types (CALLS, USES_TYPE, DEFINES), placing every co-module function pair at $d \leq 2$ via DEFINES edges. This eliminates false negatives for same-module writes but conflates module co-location with semantic dependency. Restricting BFS to CALLS edges restores intended semantics: notifications fire strictly on caller/callee relationships. This design choice should be made explicit in any deployment.

**Sensitivity to $k$.** The 14.3% false-negative rate at $k = 2$ (T13, T14) would be eliminated by increasing $k$ to 3 at the cost of additional false-positive notifications. The agent experiment confirms this trade-off is meaningful: T13's Kovex pass rate of 20% matches the baseline, showing that withheld notifications produce no benefit. Optimal $k$ is a function of codebase topology and the relative cost of false positives versus false negatives.

**Comparison to related work.** Kovex addresses the 5–10% residual semantic conflict rate of CodeCRDT [17] by operating at the semantic graph layer. It addresses AgentForge's [13] global verification overhead by restricting re-verification to the $k$-hop call neighborhood. The two approaches are complementary: Kovex issues targeted directives, and AgentForge-style sandboxed execution can serve as the verification mechanism triggered by those directives.

**Limitations.** The experiment uses a two-agent, single-codebase setting with deterministic signature changes and a CLI tool protocol that introduces noise. Real

deployments involve more concurrent agents, non-deterministic generation, and simultaneous multi-node writes. The conflict classification mechanism is Python-specific. Scaling to production multi-agent systems and additional languages remains future work.

# 6 Conclusion and Future Work

We presented Kovex, a graph-aware coordination substrate for multi-agent LLM coding systems. The core insight is that a codebase's call-graph dependency structure, represented as a queryable property graph and exposed via an MCP server, can serve as a coordination layer: agents register read dependencies on named nodes, and writes trigger targeted verification directives to all registered dependents within a $k$-hop call-graph radius.

Systems correctness evaluation on 14 tasks demonstrates 100% notification accuracy for $d \leq 2$ (mean latency 45.6 ms) and a 14.3% false-negative rate at $d = 3$ for $k = 2$. Real agent effectiveness evaluation with Claude Sonnet 4.6 shows coordinated agents achieve 80% final-code pass rate versus 0% for uncoordinated agents on tasks within the notification radius, with two tasks reaching 100% vs. 0%. The false-negative control correctly collapses to near-baseline, validating the CALLS-only BFS design.

We identify six directions for future work.

**Native SDK tool use.** The current agent experiment uses a CLI-based tool protocol that introduces noise from hallucinated tool results, capping the measured effectiveness at 80% as a conservative lower bound. Re-running the same 50 trials with the Anthropic Python SDK, which enforces strict turn structure and prevents hallucinated tool results, would either confirm the 80% figure or reveal a higher true upper bound — both outcomes are informative.

**Optimal $k$-hop study.** The choice of $k = 2$ is currently fixed without empirical justification. A systematic study running the agent experiment at $k \in \{1, 2, 3, 4\}$ across all five tasks would yield a three-way trade-off curve: as $k$ increases, pass rate and recall rise while notification volume and false-positive rate grow. Fitting a simple model to this curve, ideally across codebases of varying call-graph density, would produce an empirically grounded recommendation for $k$ as a function of codebase topology — a deployable guideline absent from the current work.

**Notification precision measurement.** The current evaluation measures recall (did Kovex notify when it should?) but not precision (did every notification correspond to a real conflict?). All current tasks are designed so every notification is a true positive by construction. Adding tasks where Agent A makes a semantically safe change — such as adding an optional parameter with a default value — and measuring how often Kovex notifies Agent B unnecessarily would complete the precision/recall picture and establish a more rigorous characterization of the false-positive rate.

**Cross-codebase generalizability.** All results are obtained on a single codebase (httpx). A reviewer will reasonably ask whether the findings generalize to codebases with different structural properties — denser call graphs, deeper inheritance hierarchies, or more cross-module dependencies. Replicating the agent experiment on a structurally distinct codebase (e.g., `requests` for a more centralized call graph, or `fastapi` for a richer type hierarchy) would establish whether the effectiveness gap holds across codebase characteristics.

**Multi-agent scaling.** The experiment is strictly two-agent. A real concurrent setting with three or more agents operating simultaneously raises questions about notification fan-out latency under load, correct handling of cascading writes (Agent B receives a directive while itself mid-write to a node that Agent C depends on), and registry consistency under concurrent `register_read` and `commit_write` calls. These scenarios require both engineering work and a new experimental design.

**Multi-language and Uses_Type traversal.** The current implementation and conflict classifier are Python-specific. Extending to TypeScript or Java would require corresponding type checkers (`tsc`, `javac`) and language-specific Tree-sitter grammars. Additionally, integrating Uses_Type edge traversal as an optional BFS mode would support type-definition-centric conflict scenarios — cases where a type field change rather than a function signature change is the source of the semantic conflict — at the cost of a higher false-positive rate that the precision study above would help quantify.

The implementation and evaluation harness are available at https://github.com/Ibrahem-al/kovex-research.

## Acknowledgments

**AI Use Disclosure.** The author used Claude (Anthropic) for drafting and editing this manuscript and for code assistance during implementation and evaluation. Perplexity AI was used to assist in identifying relevant academic literature. All scientific claims, formal definitions, algorithmic content, experimental design, and results were reviewed, verified, and approved by the author. No AI system is listed as a co-author or cited as a reference.

## References

[1] Aalto University. Consumer-driven contract testing for microservices: Practical evaluation in a distributed organization. Master's thesis, Aalto University, 2024. https://aaltodoc.aalto.fi/server/api/core/bitstreams/e035e9e7-b7a8-43c2-8c37-8020ae36dfee/content.

[2] M. Barbosa, P. Borba, R. Bonifácio, V. Lira, and G. Santos. The effect of pointer analysis on semantic conflict detection. *arXiv preprint arXiv:2507.20081*, 2025.

[3] A. Chen, Y. Yan, and D. Poshyvanyk. ACER: An AST-based call graph generator framework. *arXiv preprint arXiv:2308.15669*, 2023.

[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[5] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pages 399–407, 1989. doi: 10.1145/67544.66963.

[6] J. Ferrante, K. J. Ottenstein, and J. D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3):319–349, 1987.

[7] J. Gray, R. A. Lorie, G. R. Putzolu, and I. L. Traiger. Granularity of locks in a shared data base. In *IFIP Working Conference on Modelling in Data Base Management Systems*, pages 428–451, 1975.

[8] A. Gusarov, A. Volkova, V. Khrulkov, A. Kuznetsov, E. Maslov, and I. Oseledets. Multi-agent GraphRAG: A text-to-Cypher framework for labeled property graphs. *arXiv preprint arXiv:2511.08274*, 2025.

[9] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024. arXiv:2308.00352.

[10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. L. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

[11] A. Khatua, H. Zhu, P. Tran, A. Prabhudesai, F. Sadrieh, J. K. Lieberwirth, X. Yu, Y. Fu, M. J. Ryan, J. Pei, and D. Yang. CooperBench: Why coding agents cannot be your teammates yet. *arXiv preprint arXiv:2601.13295*, 2026.

[12] S. Krieter, J. Garcia, S. Apel, et al. Detecting higher-order merge conflicts in large software projects. In *IEEE International Conference on Software Testing, Verification and Validation*, 2020. https://aip.ifi.uni-heidelberg.de/fileadmin/papers/2020/ICST2020-merge-conflicts-final.pdf.

[13] R. Kumar, W. Ali, J. Ahmed, N. I. Ali, and S. Usman. AgentForge: Execution-grounded multi-agent LLM framework for autonomous software engineering. *arXiv preprint arXiv:2604.13120*, 2026.

[14] J. Lehvä, N. Mäkitalo, and T. Mikkonen. Consumer-driven contract tests for microservices: A case study. In *Proceedings of JAX London*, 2019. https://helda.helsinki.fi/server/api/core/bitstreams/8923de60-ede7-4b32-a31c-46c129f8197a/content.

[15] B. Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992.

[16] E. OSS. httpx: A next generation HTTP client for Python, 2021. URL https://www.python-httpx.org/. https://github.com/encode/httpx.

[17] S. Pugachev. CodeCRDT: Observation-driven coordination for multi-agent LLM code generation. *arXiv preprint arXiv:2510.18893*, 2025.

[18] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun. Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. arXiv:2307.07924.

[19] G. Santos, P. Borba, R. Bonifácio, and M. B. de Oliveira. A lightweight technique for detecting semantic conflicts with static analysis. *SSRN preprint*, 2024. doi: 10.2139/ssrn.5035182.

[20] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free replicated data types. Technical Report RR-7506, INRIA, 2011. Extended version published in *Journal of Parallel and Distributed Computing*, 2014. https://doi.org/10.1016/j.jpdc.2014.01.012.

[21] M. V. T. Thai, T. Le, D. N. Manh, H. P. Nhat, and N. D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025.

[22] I. Utkin, E. Spirin, E. Bogomolov, and T. Bryksin. Evaluating the impact of source code parsers on ML4SE models. *arXiv preprint arXiv:2206.08713*, 2022.

[23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[24] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck. Modeling and discovering vulnerabilities with code property graphs. In *IEEE Symposium on Security and Privacy*, pages 590–604, 2014. doi: 10.1109/SP.2014.44.

[25] J. Yang, A. Wettig, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.